GUIDE 63 · OPERATE

Tune a deployment for throughput

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=63-tune-a-deployment-for-throughput. This copy was made on 4 November 2026.

Time: 20 minutes to read, plus a load test of your own. **You'll end up with:** threads, CPU, logging, cache and disk settings chosen for your queries and your database's distance, a number of containers to start from, and the figures each one rests on.

Every figure here is measured on 0.43, in Performance, v4 (PostgreSQL) and Performance on MySQL, v2 - or, where those didn't rerun a block, in v3 and MySQL v1 - on one laptop with the gateway, the database and the load generator each on their own cores. Measure your own queries before you commit to a size.

1. One worker per container, more containers for more

The image runs gunicorn with **one worker** and keeps it that way: each process has its own `/metrics`, its own response cache without Redis, and its own `/events` streams. Python runs one thread at a time per process, so a worker saturates about one core. More capacity means more containers, behind a load balancer, on a shared PostgreSQL store and Redis (Run several instances).

2. Threads: by how long your calls wait

A worker's threads let it run Python for one call while others wait on the database. They help only while calls spend real time waiting. Measured, one worker, 32 callers:

THREADS	POSTGRESQL LOOKUP, SAME HOST	MYSQL LOOKUP, SAME HOST	MYSQL `STOCK_LEVEL`, SAME HOST	MYSQL `ORDER_STATUS`, 0.5 MS AWAY
1	904 req/s	**865**	630	405
2	**1,012**	823	**772**	648
4	928	722	683	**730**
8 (the default)	896	676	642	685

(PostgreSQL and the network column on 0.43; the MySQL same-host columns from v1, on code 0.43 didn't change for them.)

Past what the waiting needs, threads cost: they compete for Python's interpreter lock. On fast queries next to their database, the best setting was 13-28% faster than the default eight; with the database 0.5 ms away, four threads were best. So:

YOUR CALLS	`QUERYAPIGATE_THREADS`
Fast lookups (well under 1 ms) on a database on the same host or rack	`2`
Queries of a few ms, or a database a network hop away	`4` - `8`
Slow reports (tens of ms and up), or a warehouse or Trino across a network	`8` - `16`, while CPU allows

The waiting limit of one worker is **threads ÷ (query time + round trips × network round-trip time)** calls a second. From 0.43 a PostgreSQL call makes 1 round trip to its database and a MySQL call 2, plus 3 to a PostgreSQL store, none with the metadata cache (section 4). A 20 ms report query at 0.5 ms away, with 8 threads: 8 ÷ (20 + 4 × 0.5 ms) = about 360 a second - more threads would raise it, as long as the CPU figure in section 7 isn't lower.

MySQL across a network before 0.43: more than two threads lost throughput - mysql-connector held Python's interpreter lock while it pinged the server, once per call (ADR 0027). On 0.43 there's no ping, and four threads serve 46% more than 0.42 could with any setting.

The pool of idle database connections follows the threads (`QUERYAPIGATE_POOL_SIZE` defaults to them), and the main server's `/events` streams never take more than half of them.

yaml

```yaml
# Docker Compose
    environment:
      QUERYAPIGATE_THREADS: "2"
```

yaml

```yaml
# Helm
config:
  QUERYAPIGATE_THREADS: "2"
```

3. CPU: about 1.25–1.5 cores per container

At full load one worker used **1.1–1.3 cores**: one for Python, the rest for its threads' I/O (PostgreSQL 1.2, MySQL 1.1–1.3). A CPU limit below that throttles it, and the throttling shows up as latency, not as an error.

	REQUEST	LIMIT
A small or shared cluster (the Helm chart's default)	`250m`	none
Serving at capacity	`1`	`1.5`, or none

yaml

```yaml
# Helm
resources:
  requests: {cpu: "1", memory: 256Mi}
  limits: {cpu: 1500m, memory: 1Gi}
```

On one host, give each container its own physical core. Measured: four workers on one hyperthread of each of four cores served 2,250 calls a second; on both hyperthreads of two cores, 1,807 - a core's second hyperthread is worth about 0.6 of a core. Four workers allowed all eight hyperthreads of four cores did worse than on four (1,756), because

the scheduler sometimes put two on one core.

4. The metadata cache: +10-22%, and most of the network's cost

`QUERYAPIGATE_METADATA_CACHE_TTL=5` keeps the key, saved query and connection a call reads from the store for 5 seconds instead of reading them on every call. Measured on 0.43: +15-19% on PostgreSQL, +10-18% on MySQL, +15-22% on a PostgreSQL store with 1-8 workers, and the store's load cut to a third. Across a network it matters most: with PostgreSQL and the store 0.5 ms away, one call took 4.39 ms without it and **1.60 ms** with it - one round trip instead of four.

The cost: a change made on **another** instance (or with the CLI) takes up to the TTL to arrive there - a revoked key keeps working on the other instances for that long. A change made through the same instance is seen at once. With one instance, turn it on; with several, decide whether a 5-second revocation delay is acceptable (the metadata cache).

5. Logs: off where something else already logs

SETTING	WHAT IT WRITES	COST MEASURED	TURN OFF WHEN
`QUERYAPIGATE_ACCESS_LOG=off`	gunicorn's access line per request, on stdout	6-7% of throughput	the proxy or load balancer logs requests
`QUERYAPIGATE_LOG_REQUESTS=off`	QueryAPIGate's own request and query lines (INFO)	another 14%	the same - errors, warnings, slow queries and agent writes are still logged

Both off: +21% on one worker (MySQL v2). With the metadata cache on, logging is the largest single cost left in a call: a quarter of the CPU samples on PostgreSQL.

Turn the access log off with the setting, not by sending it to `/dev/null`: that still formats and writes every line. At these rates the logs grow by gigabytes an hour - 6.6 GB in a five-hour MySQL run.

6. Disk: the event log

Every call is recorded in the store's event log (ADR 0020), kept for 24 hours by default: **about 570 bytes a call** on a SQLite store, more on PostgreSQL.

SUSTAINED CALLS A SECOND	EVENT LOG AFTER 24 HOURS
50	about 2.5 GB
500	about 25 GB
2,000	about 100 GB

`QUERYAPIGATE_EVENTS_RETENTION_HOURS` shortens it, and the disk with it. It is also how far back a `queryapigate events` client can resume after a disconnect, so keep it above the longest outage a client must ride out.

7. How many containers

Per worker, at 75% of measured capacity, where p99 stays at about 4–5 ms:

	PLAIN	WITH THE METADATA CACHE	CACHE, BOTH REQUEST LOGS OFF
PostgreSQL, fast OLTP calls	670 calls a second	780	-
MySQL, `order_status`-class calls	500	545	650

(0.42's were 550 / 650 and 440 / 490.) For a sense of the ceiling: one laptop with six performance cores, twelve workers of two threads and both logs off, served about 3,500 multi-database MySQL joins a second, sustained.

Containers = the larger of (your calls a second ÷ that figure) and (your calls a second ÷ the waiting limit from section 2), **plus one** to lose without falling over. Every millisecond of Python a call costs beyond these simple lookups lowers the CPU figure; masked keys and large pages cost more.

Then load-test your own queries at 75% of what you plan to serve, and watch p99, not the average: it holds flat to about 75% and triples by 90%.

What hasn't been measured yet

- **Containers on separate hosts.** The scale-out figures are workers sharing one laptop's cores; separate hosts should scale better, by an amount not yet measured.
- **The light-load p99.9.** At a quarter of capacity, a few calls in a thousand take about 30 ms, in bursts every few seconds; p99 is 3.4 ms. The history writer and, mostly, the garbage collector are ruled out; the cause isn't found yet (MySQL v2).